

Fundação Getúlio Vargas
Estrutura de Dados

Relatório Lista 4

Alunos:

Ana Beatriz Botacim Rodrigues — 251708010

Larissa Conerado Ribeiro — 251708026

Professor:

Matheus Telles Werner

8 de junho de 2026

Conteúdo

1	Introdução	2
2	Representação da Trie	2
3	Funcionamento dos métodos	2
3.1	Função insert	2
3.2	Função contains	2
3.3	Função autocomplete	2
4	Análise de custo	2
4.1	Função insert	3
4.2	Função contains	3
4.3	Função autocomplete	3
5	Comparação e uso de memória	3

1 Introdução

Este trabalho apresenta uma solução em C++ para autocomplete e busca de jogos em uma Trie.

2 Representação da Trie

Para a organização da estrutura, foi definido que o array `children` teria como elementos 36 ponteiros: sendo 26 representando as letras do alfabeto latino e 10 para os algarismos arábicos. Através do código `ascii`, pudemos fazer a tradução do índice para o ponteiro correspondente da letra ou número. Primeiramente, os títulos e prefixos inseridos são transformados, caractere a caractere, em seus valores respectivos na tabela `ascii`. A cada um, verificamos: se o valor `ascii` está no intervalo `[97, 122]`, ou seja, se é uma letra minúscula, não precisa ser transformada; se está no intervalo `[65, 90]` é uma letra maiúscula, ao qual somamos 32 a fim de transformá-la em letra minúscula; para o intervalo `[48, 58]` temos um número, portanto, o valor é mantido; por fim, se o valor for diferente desse, significa que não será considerado como parte da busca.

3 Funcionamento dos métodos

3.1 Função `insert`

Recebe um ponteiro para um jogo e transforma seu nome em uma chave de pesquisa válida para adicioná-la na trie. Percorre os níveis da árvore começando pela raiz, e, em cada nível (que corresponde a um caractere do nome) verifica a existência de nó filho correspondente ao caractere atual – o valor inteiro positivo ao subtrair 97 do valor `ascii` do caractere, ou, caso negativo, o valor encontrado ao subtrair 48 do valor `ascii` do número – caso não exista, criamos o ponteiro, e seguimos a busca pelo próximo caractere a partir do nó recém criado. Caso o ponteiro buscado já exista, a função segue descendo mais um nível na Trie. Após verificar todos os caracteres, definimos o valor de `isEndOfTitle` do último nó para `true` e informamos o jogo correspondente. Retornará `true` após inserir o jogo.

3.2 Função `contains`

Assim como `insert`, percorre a Trie nível a nível verificando a existência dos nós correspondentes aos caracteres formadores da palavra. Ao passar por todos os caracteres, se o último nó correspondente tiver `isEndOfTitle` com o valor `true` a função retorna `true`, caso contrário, retorna `false`.

3.3 Função `autocomplete`

Ao receber um prefixo como argumento, o transforma em uma chave válida. Se o valor do parâmetro k , definido no momento de execução do código, for válido ($k > 0$), percorre totalmente o prefixo para realizar, a partir do último caractere inserido, uma busca em largura.

Ao chegar no fim do prefixo, criamos um vetor auxiliar com os nós a serem visitados. Faremos isso por níveis, então, adicionamos ao vetor todos os caracteres possíveis no próximo nível – os filhos dos nós do nível atual – os quais iremos verificar se possuem filhos, que serão adicionados ao vetor posteriormente, e assim por diante.

Ao fazer a análise de possíveis nós em um dado nível, removemos os nós do nível anterior do vetor auxiliar.

Para cada nó que visitamos, verificamos `isEndOfTitle`: caso seja `true`, adicionamos o nome do jogo correspondente ao nosso vetor de jogos a serem retornados.

Ao percorrer todos os nós até o fim da árvore, o vetor de jogos é ordenado de acordo com as regras da estrutura, por meio de um algoritmo modificado de insertion sort. Retorna um vetor de ponteiros do tipo jogo.

4 Análise de custo

Variáveis:

- * `'L'`: tamanho do título após conversão para chave de busca;
- * `'p'`: tamanho do prefixo após conversão para chave de busca;
- * `'s'`: número de nós visitados na subárvore do prefixo;
- * `'r'`: número de jogos encontrados para o prefixo buscado.

4.1 Função insert

Para cada jogo, iremos transformar seu nome completo em uma chave válida através da função `toSearchKey`, que realiza a verificação de cada caractere pertencente à string informada – $O(L)$. Após isso, para cada caractere, iremos adicionar um ponteiro caso este ainda não tenha sido criado na Trie. Como verificamos todos os caracteres da chave válida, somamos ao custo computacional L . Dessa forma, temos custo aproximado $2L$, correspondente a $O(L)$.

4.2 Função contains

Novamente utilizamos a função `toSearchKey`, que acumula complexidade $O(L)$. Após isso, para cada caractere iremos verificar se o ponteiro para os caracteres do jogo buscado existem, sequencialmente. Como verificamos todos os caracteres da chave válida, somamos ao custo computacional L . Assim, temos custo aproximado $2L$, correspondente a $O(L)$.

4.3 Função autocomplete

Primeiramente, transformamos o prefixo adicionado em uma chave válida com custo computacional p , a qual vamos percorrer completamente, somando p operações. Ao chegarmos no último caractere do prefixo, vamos considerar todas as possíveis ramificações, verificando assim 36 espaços em `children` para cada caractere. Ao passo que verificamos todos os possíveis s nós, adicionamos os r jogos ao vetor, custo r .

Chegamos em um custo final aproximado de $p + p + s * 36 + r$. Como temos $r \leq s$, pois não é possível ter mais jogos do que nós nesta modelagem, o custo é da ordem de grandeza $O(\max\{p, s\})$.

5 Comparação e uso de memória

Consideremos, para essa análise, n sendo o número total de jogos do sistema.

Ao considerar um array ordenado como a solução ingênua para nosso problema, o faremos de forma a inserir os jogos em ordem alfabética. Para cada jogo inserido, teríamos que verificar todos os caracteres de todos os jogos já adicionados. Dessa forma, teríamos uma complexidade dependente da quantidade de caracteres do nome de cada jogo e do tamanho do array de jogos já inseridos. Ao adicionar um jogo, seria necessário realocar todos os jogos que vêm após ele na ordem alfabética. Isso custa $O(n)$ para cada jogo adicionado.

Para o retorno do `autocomplete` e do `contains`, todos os jogos seriam verificados para encontrar os índices referentes aos jogos de mesmo prefixo, o que acaba dependendo da quantidade de jogos encontrados a cada caractere do prefixo. No pior dos casos, custaria $O(n)$ para a verificação de cada caractere do prefixo, mais r verificações - uma para cada jogo encontrado - de encerramento do título após o prefixo o que acumula custo computacional aproximado da ordem de $O(p \times n + r) = O(p \times n)$ para o `contains`. No caso do `autocomplete`, ainda seria necessária a ordenação com, em um dos casos ótimos, o algoritmo de quick sort. O custo total, portanto, ficaria na ordem de $O(p \times n + r \log r)$. Como $r \leq n$, temos o pior custo na ordem de $O(n \times (p + \log n))$.

Percebe-se que a solução proposta por nós no presente trabalho se mostrou muito mais eficiente que a ingênua, tendo custo linear em L para quase todas as funções, e linear no $\max\{p, s\}$ para o `autocomplete`, que ainda é infinitamente melhor que a solução ingênua, na análise assintótica.

Quando compartilhamos os prefixos, acabamos reduzindo o número de nós porque, para encontrar r dados jogos, percorreremos os mesmos primeiros p nós, em vez de ter que fazer, em cada um dos n nós - no caso da solução ingênua, em que cada filho ocupa um espaço do array -, uma verificação se o nome é compatível com o fornecido para busca. Essa estrutura é boa porque acaba agrupando os jogos com nomes semelhantes, em relação ao prefixo, que é justamente a forma solicitada para realizar uma busca por nomes.

O tamanho do alfabeto afeta diretamente o tamanho ocupado por cada nó porque cada nó guarda consigo mesmo um array do tamanho do alfabeto, que contém os ponteiros de seus - possíveis - filhos.